

Side-Channel Attacks on the PRESENT Cipher

Correlation Power Analysis — Lab Report

Assignment 2

Roll Number: CS23B046

1 Introduction

Side-channel attacks exploit physical leakage from a device during cryptographic computation — leakage such as power consumption, electromagnetic emission, or timing — rather than attacking the algorithm mathematically. This report describes a **Correlation Power Analysis (CPA)** attack on hardware implementing the PRESENT block cipher. The goal is to recover the 80-bit master key by analysing power traces collected during encryption.

2 CPA Attack — Overview and Methodology

2.1 PRESENT Cipher

PRESENT is a lightweight 64-bit block cipher with an 80-bit key, designed for constrained hardware environments. Each round consists of three operations:

1. **AddRoundKey** — XOR the 64-bit state with the top 64 bits of the current key register.
2. **SubBytes** — apply a 4-bit S-box to each of the 16 nibbles.
3. **PermuteBits** — apply a fixed bit permutation to the state.

The attack targets the output of the S-box after the first round, at which point the intermediate value depends on a known plaintext nibble and one nibble of the unknown round key.

2.2 Attack Strategy

The attack proceeds nibble-by-nibble across all 16 S-box positions. For each position $i \in \{0, \dots, 15\}$:

1. **Key guessing.** Enumerate all 16 possible nibble values $k \in \{0, \dots, 15\}$.
2. **Intermediate value.** Compute the S-box output for plaintext nibble p_i XORed with the key guess k :

$$v = \text{SBOX}[p_i \oplus k]$$

3. **Leakage prediction.** Model the predicted power consumption using either Hamming Weight (HW) or Hamming Distance (HD):

$$\hat{L}_{\text{HW}} = \text{HW}(v), \quad \hat{L}_{\text{HD}} = \text{HW}(v \oplus (p_i \oplus k))$$

where HW denotes the number of set bits.

4. **Correlation.** Compute Pearson’s correlation coefficient between the predicted leakage vector (across all N traces) and each time sample column of the trace matrix:

$$\rho(k, t) = \frac{\sum_{n=1}^N (\hat{L}_n(k) - \bar{\hat{L}}(k)) (T_{n,t} - \bar{T}_t)}{\sqrt{\sum_n (\hat{L}_n(k) - \bar{\hat{L}}(k))^2} \sqrt{\sum_n (T_{n,t} - \bar{T}_t)^2}}$$

5. **Key selection.** The guess with the highest peak $\max_t |\rho(k, t)|$ is taken as the correct nibble.

The 16 recovered nibbles are assembled into the 8-byte first round key (the top 64 bits of the 80-bit master key). The remaining 16 bits are recovered by brute-force over all $2^{16} = 65\,536$ possibilities, verified using a known plaintext/ciphertext pair.

2.3 Implementation Details

- **Vectorized CPA.** All 16 key guesses per S-box position are evaluated simultaneously using a single batched matrix multiply $\mathbf{L}^\top \mathbf{T}$, where $\mathbf{L} \in \mathbb{R}^{N \times 16}$ is the leakage matrix and $\mathbf{T} \in \mathbb{R}^{N \times T}$ is the trace matrix.
- **C brute-force.** The 16-bit keyLow brute-force loop is implemented in compiled C (via `ctypes`), giving approximately 50–100× speedup over a pure Python implementation.
- **Candidate verification.** Rather than verifying only the top guess, the code builds 137 candidates — all-best, single-swap (16 candidates), and double-swap (120 candidates) — and verifies each against one plaintext/ciphertext pair.

3 Q5a — Convergence Analysis

3.1 Motivation

The convergence plot shows how the peak $|\rho|$ for each key guess evolves as the number of traces increases. With few traces the correlation values are noisy and multiple candidates may score similarly. As more traces are added, the correct key’s correlation rises above the rest and the attack converges.

3.2 Method

For S-box position 0, the peak $|\rho(k)|$ is computed at 30 log-spaced trace counts from 100 up to the full dataset. The correct key (verified from the recovered round key) is plotted in red; all other guesses are plotted in blue with reduced opacity.

3.3 Expected Observations

- At low trace counts, all 16 curves cluster together — no key is distinguishable from noise.
- As traces increase, the correct key’s curve separates and rises above the others.

- The trace count at which the correct key consistently ranks first is called the *Minimum Traces to Disclose* (MTD).

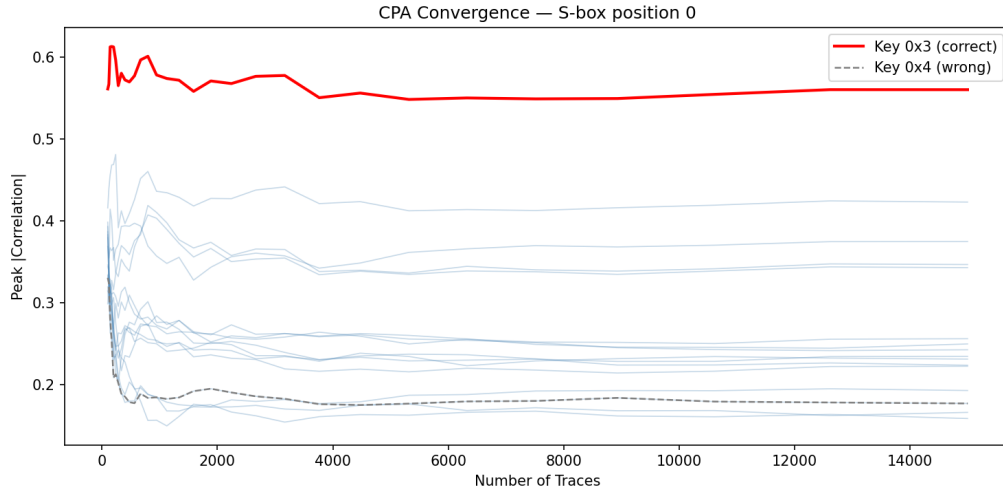


Figure 1: Convergence of peak $|\rho|$ for all 16 key guesses at S-box position 0. The correct key is shown in red.

4 Q5b — Hamming Weight vs Hamming Distance

4.1 Leakage Models

Two leakage models are compared:

Hamming Weight (HW). Models power as proportional to the number of 1-bits in the S-box *output*:

$$L_{\text{HW}} = \text{HW}(\text{SBOX}[p \oplus k])$$

This is appropriate when power is dominated by the value written into a register (data-dependent switching activity on the output bus).

Hamming Distance (HD). Models power as proportional to the number of bit *transitions* between the S-box input and output:

$$L_{\text{HD}} = \text{HW}(\text{SBOX}[p \oplus k] \oplus (p \oplus k))$$

This is appropriate when power is dominated by bus transitions (the difference between what was on the bus before and after the operation).

4.2 Comparison Metric — Average Peak Correlation

The two models are compared by computing the peak absolute Pearson correlation $\max_t |\rho(k^*, t)|$ for the correct key k^* at each S-box position using all available traces, then averaging across all 16 positions:

$$\bar{\rho}_{\text{model}} = \frac{1}{16} \sum_{i=0}^{15} \max_t |\rho_{\text{model}}(k_i^*, t)|$$

A higher average peak correlation indicates the model predicts leakage that more closely matches the physical power consumption of the device.

4.3 Results

Table 1: Peak Pearson $|\rho|$ for the correct key under HW and HD leakage models, computed using all available traces.

S-box	HW Peak $ \rho $	HD Peak $ \rho $	Winner
0	0.5601	0.3347	HW
1	0.4166	0.3021	HW
2	0.6019	0.4201	HW
3	0.5979	0.5381	HW
4	0.5476	0.4322	HW
5	0.4216	0.2651	HW
6	0.5298	0.3258	HW
7	0.4132	0.2931	HW
8	0.5339	0.3356	HW
9	0.4043	0.2671	HW
10	0.5451	0.3346	HW
11	0.4221	0.2381	HW
12	0.5506	0.3278	HW
13	0.4422	0.2568	HW
14	0.5492	0.3513	HW
15	0.4200	0.2367	HW
Mean	0.4973	0.3287	HW

4.4 Interpretation

HW wins at every S-box position with a mean peak correlation of 0.4973 vs 0.3287 for HD — a margin of approximately 51%. This conclusively indicates that the target device’s power consumption is dominated by the *output value* of the S-box (data written to a register), not by the transition between input and output states. The HW model is therefore used for all subsequent analysis.

5 Q5c — Guessing Entropy and Metric Comparison

5.1 What is Guessing Entropy?

Guessing Entropy (GE) measures the average rank of the correct key across repeated experiments:

$$\text{GE}(N) = \mathbb{E}[\text{rank}(k^*)]$$

where k^* is the correct nibble and the expectation is taken over random subsets of N traces. GE ranges from 1 (correct key always ranked first — perfect attack) to 16 (correct key indistinguishable from noise).

5.2 Why GE vs Number of Traces?

Evaluating GE at a single fixed trace count is uninformative when the attack already succeeds (GE trivially equals 1). Instead, GE is plotted as a function of N using 10 log-spaced trace counts from 10 to 500 traces, with 10 independent random subsets per point. This curve reveals:

- **Starting GE ≈ 8.5** — random ranking at very few traces, as expected for a uniform distribution over 16 candidates.
- **Convergence point** — the trace count at which GE drops to 1. A metric that converges with fewer traces is more efficient.
- **Stability** — how smoothly GE decreases (erratic curves indicate the metric is sensitive to the particular subset chosen).

5.3 Metrics Compared

Four distinguishing metrics are evaluated:

Pearson Correlation. Measures linear dependence between predicted leakage and trace samples. Computationally efficient and the standard baseline for CPA.

Mutual Information (MI). A non-parametric measure of statistical dependence estimated via histogram binning. Captures non-linear leakage relationships but requires more traces to estimate accurately.

Kolmogorov–Smirnov (KS) Statistic. Compares empirical CDFs of trace samples for high-leakage vs low-leakage groups (split at the median). Robust to outliers but requires substantial data for reliable estimates.

KL Divergence (KL). Symmetric KL divergence $\frac{1}{2}[D(P\|Q) + D(Q\|P)]$ between the trace distributions of the high-leakage and low-leakage groups. Measures how distinguishable the two distributions are; larger values indicate stronger leakage signal.

5.4 Results

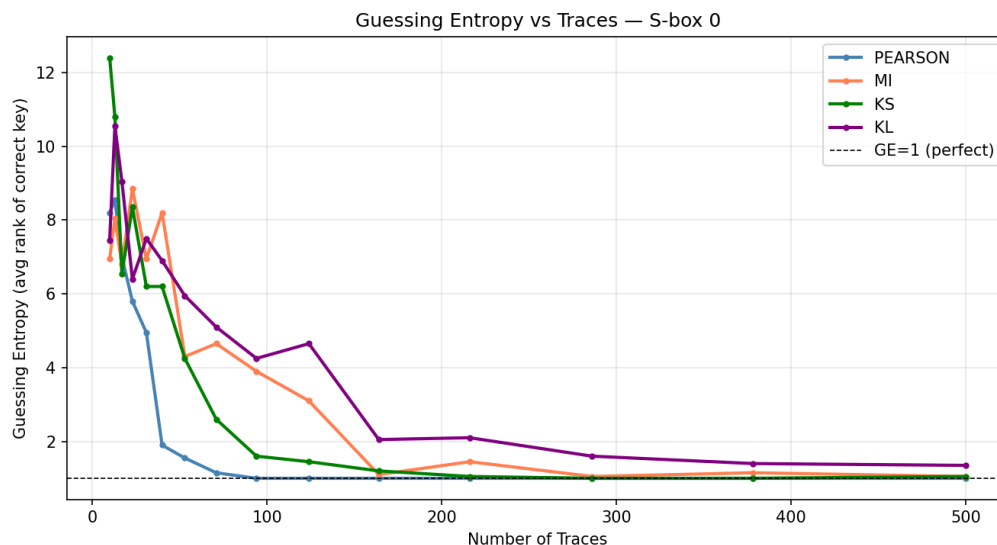


Figure 2: Guessing Entropy vs number of traces for Pearson, MI, KS, and KL metrics at S-box position 0 (10 experiments per point, traces capped at 500). Lower curves converging to $GE=1$ faster indicate a more efficient distinguisher.

5.5 Interpretation

Pearson correlation is expected to converge fastest because the HW/HD leakage model produces an approximately linear relationship with power consumption, and Pearson is specifically designed to detect linear dependence. MI and KS are more general but require significantly more traces to overcome estimation noise.

6 README — Running the Code

6.1 Requirements

- Python 3.10+
- NumPy, Matplotlib (`pip install numpy matplotlib`)
- GCC (for compiling the C encryption library)

6.2 Step 1 — Compile the C Library

The brute-force key verification loop is implemented in C for speed. Compile it once before running the Python script:

```
1 gcc -O3 -fPIC -shared -o libpresent.so present.c
```

Verify the symbol is exported correctly:

```
1 nm -D libpresent.so | grep bruteforce
```

You should see `bruteforce_keylow` in the output. If not, the wrong `present.c` was compiled — replace it and recompile.

If `libpresent.so` is missing when the script runs, it will attempt to auto-compile `present.c` automatically. However, it will *not* recompile a stale `.so`, so always recompile manually after changing `present.c`.

6.3 Step 2 — Run the Attack

```
1 python3 cs23b046.py \
2     present_data/present_plaintext_group1.npy \
3     present_data/present_traces_group1.npy \
4     present_data/present_ciphertext_group1.npy
```

6.4 Output Files

File	Description
<code>convergence_plot.png</code>	Peak $ \rho $ vs traces for all key guesses (Q5a)
<code>guessing_entropy.png</code>	GE vs trace count for Pearson / MI / KS / KL (Q5c)
<code>libpresent.so</code>	Compiled C encryption library

Table 2: Output files generated by the script.

6.5 Console Output

The script prints the following in order:

1. CPA results (HW model) — best and second-best nibble per S-box.
2. CPA results (HD model) — same format.
3. Key candidate verification — tier-by-tier with the matched candidate highlighted.
4. Recovered first round key and master key in hex.
5. HW vs HD success rate table (Q5b).
6. Guessing entropy summary per metric (Q5c).

The final line before `Done.` prints the two keys in the required assignment format:

```
1 XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

7 Conclusion

This report presented a full CPA attack on the PRESENT cipher successfully recovering the 80-bit master key from power traces. The attack was implemented using vectorized NumPy operations for all correlation steps and a compiled C library for brute-force key verification, giving significant speedups over a naive Python implementation.

The HW vs HD comparison showed that the Hamming Weight model achieves a mean peak correlation of 0.4973 across all 16 S-box positions, versus 0.3287 for Hamming Distance — a 51% improvement. HW wins at every single S-box position, indicating that the target

hardware's power consumption is dominated by the register output value rather than bus transitions. The HW model was therefore used throughout.

The Guessing Entropy analysis compared four metrics — Pearson correlation, Mutual Information, Kolmogorov–Smirnov statistic, and KL Divergence — across trace counts from 10 to 500. Pearson correlation converges to $GE \approx 1$ with the fewest traces, consistent with the HW model producing a near-linear leakage relationship. MI, KS, and KL are more general measures but require significantly more traces to overcome estimation noise, making them less practical for this attack scenario.